

CMPE 260 – Homework 2 Solutions

Your Name

March 27, 2025

Introduction

These answers focus on names, bindings, and scopes in programming languages. Make sure to rephrase, adjust, or expand them as needed to reflect your own understanding.

Question 1 – Name Binding and Scope Resolution

Pseudocode:

```
x = 10
```

```
def outer():  
    x = 5  
    def inner():  
        x = 3  
        print(x)  
    inner()  
    print(x)
```

```
outer()  
print(x)
```

1) Output in a statically scoped (lexically scoped) language:

- `inner()` has its own local `x = 3` and prints 3.
- After `inner()`, `outer()` prints `x = 5`.
- Finally, the global `x = 10` is printed outside `outer()`.

Hence, the output is:

3 (*from inner()*) 5 (*from outer()*) 10 (*global*).

2) Output in a dynamically scoped language:

Under typical dynamic scope rules, `x = 3` in `inner()` may override the `x` in `outer()`. Hence the second print might also produce 3, not 5. The final print outside remains 10. Thus:

3, 3, 10.

3) Draw the static and dynamic scope chains:

- **Static (lexical) chain:**

- (a) Global scope: `x = 10`.
- (b) `outer()` scope: `x = 5`, enclosed by global scope.
- (c) `inner()` scope: `x = 3`, enclosed by `outer()`.

References to `x` in `inner()` first check `x = 3`, then `x = 5`, then `x = 10` if needed.

- **Dynamic chain:**

- (a) `inner()` is called by `outer()`, which is called by the global environment.
- (b) Under dynamic scoping, `x` is found by traversing the call chain at run time: `inner()` → `outer()` → global.

4) Modify code so that `inner()` accesses `x` from `outer()` explicitly:

In a Python-like language:

```
x = 10

def outer():
    x = 5
    def inner():
        nonlocal x
        x = 3
        print(x)
    inner()
    print(x)

outer()
print(x)
```

The `nonlocal` keyword (or equivalent in other languages) forces `inner()` to use the `outer()`-scope `x`, rather than creating a new local variable.

Question 2 – Binding and Lifetime of Variables

C-style pseudocode:

```
int x = 5;

void foo() {
    static int x = 10;
    int y = x;
    x = x + 5;
}
```

```
int main() {
    foo();
    foo();
    printf("%d\n", x);
}
```

1) Value of x printed at the end of execution:

There are two x variables:

- A global x = 5.
- A static local x = 10 in foo() that persists between calls.

Each time we call foo(), the static local x is incremented by 5. After two calls, it becomes 20. However, the global x remains 5 and is printed in main(), so the output is 5.

2) Difference between static binding and stack-dynamic binding:

- `static int x` is allocated once (in static storage) and persists across function calls.
- A normal function-local variable (stack-dynamic) is created on the stack on each function invocation and destroyed upon returning from the function.

3) Modify the code so x in foo() is no longer retained:

Replace

```
static int x = 10;
```

with

```
int x = 10;
```

Then it will re-initialize on each call to 10 and not accumulate across calls.

4) Extend the code by adding a heap-allocated variable:

```
void foo() {
    static int x = 10;
    int y = x;
    x = x + 5;

    int* ptr = malloc(sizeof(int));
    *ptr = 100;
    // Use ptr ...
    free(ptr); // avoid memory leak
}
```

The variable `ptr` resides on the stack, but the memory it points to (`*ptr`) lives on the heap.

Question 3 – Type Binding and Type Inference

Python-style pseudocode:

```
x = 10
y = "20"
z = x + y
```

1) What happens in a statically typed language?

A statically typed language typically forbids adding an integer and a string without explicit conversion, causing a compile-time error.

2) What happens in a dynamically typed language?

In Python, attempting `x + y` with `x = 10` and `y = "20"` will raise a `TypeError` unless explicitly converted (e.g., `int("20")`) or if the language's semantics automatically convert them (some dynamic languages might perform implicit conversions differently).

3) Modify the code to enforce type consistency using type annotations:

```
x: int = 10
y: str = "20"
z: int = x + int(y)
```

In a strictly typed language (e.g., Java):

```
int x = 10;
String y = "20";
int z = x + Integer.parseInt(y);
```

4) Advantages and disadvantages of implicit vs. explicit type binding:

- **Implicit:**
 - (+) Less boilerplate, more concise.
 - (-) Possible confusion if the inferred type is not what the programmer expected.
- **Explicit:**
 - (+) Clarity of types, catches errors early in large codebases.
 - (-) More verbose, can slow down rapid prototyping.

5) Example where type inference leads to ambiguity:

If a function returns something unknown:

```
var result = someFunction();
-- or --
auto result = someFunction();
```

The compiler might infer a type that was not intended. To fix it, you add an explicit annotation or cast (e.g., `int` or `String`).

Question 4 – Ambiguity in Scope Resolution

JavaScript-style pseudocode:

```
var x = 5;

function foo() {
  var x = 10;
  function bar() {
    console.log(x);
  }
  bar();
}

function baz() {
  console.log(x);
}

foo();
baz();
```

1) Output if JavaScript uses static scope:

Lexically, **bar** sees **x = 10** from **foo()**, and **baz** sees **x = 5** (global). Output:

10, 5.

2) Output if JavaScript uses dynamic scope:

Even with dynamic scoping, in this snippet (as written), **bar** would find **x = 10** from **foo()**'s call environment, and **baz** sees the global 5. The output is again 10, 5.

3) Modify the code so **bar() refers to the global **x** regardless of scope model:**

Rename the local variable in **foo()**:

```
function foo() {
  var y = 10;    // no longer shadow 'x'
  function bar() {
    console.log(x); // refers to global x = 5
  }
  bar();
}
```

4) Introduce a **let statement:**

```
function foo() {
  let x = 10;    // block-scoped
  function bar() {
```

```

        console.log(x);
    }
    bar();
}

```

Using `let` prevents hoisting-related pitfalls that come with `var`.

Question 5 – Explicit and Implicit Type Binding

Pseudocode:

```

# Explicit type binding
int x = 10

```

```

# Implicit type binding
y = 20

```

1) Difference between explicit and implicit type binding:

- **Explicit:** You write the type in the declaration (e.g., `int x = 10`).
- **Implicit:** The language infers the type from usage (e.g., `y = 20` in Python).

2) Example where implicit type binding leads to an error:

If a language infers `y` is an integer, but the programmer later uses `y` as a string, it causes an error. For instance:

```

y = 20          # Inferred as int
y = "Hello"     # Type error if the language enforces consistency

```

3) Rewrite using type inference:

In modern languages like Kotlin or C++ with `auto`:

```

auto x = 10;    // x is an int
auto y = 20;    // y is an int

```

4) How implicit type binding affects type checking and safety:

It can reduce verbosity but also obscure the actual type if it's not evident. If the compiler infers incorrectly or the code's intent is unclear, bugs can occur.

Question 6 – Heap vs Stack Allocation

C++-style pseudocode:

```
int* p = new int(10);

void foo() {
    int x = 20;
    int* y = new int(30);
}
```

1) Which variables are allocated on the stack vs. the heap?

- `x = 20` is on the stack (inside `foo`).
- `p` and `y` are pointers. The actual `new int(...)` calls allocate `int` values on the heap. The pointer variables themselves (`p`, `y`) live on the stack or in static/global area if declared globally.

2) Deallocation process:

- Stack variables (like `x` in `foo()`) are automatically popped off when `foo()` returns.
- Heap variables (`new int(10)` or `new int(30)`) must be manually freed using `delete` in C++ (or `free` in C).

3) Extend the code to create a memory leak and explain how to fix it:

```
int* p = new int(10);
// ... no 'delete p' ...
// => memory leak if the program ends or p goes out of scope
```

To fix:

```
delete p; // Deallocation
```

4) Proper deallocation with a destructor or delete:

In modern C++, prefer smart pointers:

```
#include <memory>
std::unique_ptr<int> p = std::make_unique<int>(10);
// automatically freed when p goes out of scope
```

Question 7 – Hoisting and Block Scope

JavaScript pseudocode:

```
console.log(x);
var x = 10;

if (true) {
```

```
    let y = 20;
  }
```

```
console.log(y);
```

1) Behavior when this code is executed:

- `console.log(x);` prints `undefined`, because `var x` is hoisted but not yet assigned.
- `let y = 20;` is block-scoped; outside the `if`, `y` is not visible, so `console.log(y);` causes a `ReferenceError`.

2) Concept of hoisting:

`var` declarations are effectively moved to the top of the scope, but assignments remain in place. `let` and `const` are also hoisted but remain in the so-called “Temporal Dead Zone” until their actual declaration line is reached.

3) Modify the code to avoid the error using `let` or `const`:

```
let x = 10;
console.log(x); // prints 10

if (true) {
  let y = 20;
  console.log(y); // prints 20
}
```

Now `y` is used only within its block.

4) Why `let` is preferred over `var` in modern JS:

`let` is block-scoped, avoids many subtle errors, and does not pollute the function or global scope. It also avoids issues with hoisting where a variable is accessible before declaration.

Question 8 – Referencing Environment and Dynamic Scoping

Lisp-style pseudocode:

```
(define x 5)

(define (foo)
  (define x 10)
  (bar))

(define (bar)
  (display x))

(foo)
```


1) Output under dynamic scoping:

When `foo()` runs, it defines a new local `x = 10` and calls `bar()`. Under dynamic scoping, `bar()` finds `x = 10`, so it displays 10.

2) Output under static (lexical) scoping:

`bar()` was defined in the global environment, so it uses the global `x = 5`, ignoring `foo()`'s local definition. It displays 5.

3) Modify code so `bar` accesses the global `x` under dynamic scoping:

Rename the local variable:

```
(define (foo)
  (define local-x 10)
  (bar))
```

Then `bar()` sees `x = 5` globally.

4) Closures for a functional language solution:

In a purely lexical-scoping functional language (like Scheme), each function “captures” the environment in which it was defined. If `bar` should always see the global `x`, we can define it in the global scope or design it so it closes over that variable specifically.

Question 9 – Named Constants and Static Typing

C++-style pseudocode:

```
const int x = 5;
```

```
void foo() {
    x = 10;
}
```

1) Compilation result:

You will get a compile-time error because `x` is a constant and cannot be reassigned.

2) Difference between `const` and `readonly` in C#:

- `const` is a compile-time constant; must be assigned when declared.
- `readonly` can be assigned in the declaration or a constructor, but not changed afterward. It is a runtime constant.

3) Make `x` modifiable while retaining compile-time type checking:

Simply remove `const`:

```
int x = 5;
```

Now `x` can be reassigned as needed.

4) How named constants improve program reliability and readability:

They avoid “magic numbers,” reducing errors if a value needs changing in multiple places. They clearly communicate that the value should not vary and thus reduce accidental modifications.

Final Notes. These explanations address fundamental topics: static vs. dynamic scoping, variable lifetime (stack, heap, static), explicit vs. implicit typing, hoisting, and named constants. Use them as a study guide, and adapt them to your own words for clarity and completeness.